

Interpreter - Phiên dịch

Mẫu thiết kế - SE401.Q21

23521224 Trương Hoàng Phúc

23521140 Lê Minh Phát

22520243 Ya Đạt

1	Tên & Phân loại	4
2	Mục đích & Ý định	6
3	Vấn đề	8
4	Giải pháp	10
5	Cấu trúc	12
6	Luồng hoạt động	14
7	Mã nguồn minh họa	16
8	Ứng dụng thực tế	21
9	Trường hợp áp dụng	23
10	Ưu nhược điểm	25
11	Cách áp dụng	27
12	So sánh với các mẫu khác	29
13	Các mẫu liên quan	31

14	Lỗi thường gặp	33
15	Tóm tắt	35

1 Tên & Phân loại

- Tên gọi: Interpreter, phiên dịch
- Tên khác: Translator, Parser
- Phân loại: Behavioral Pattern (Mẫu ứng xử)

Xác định cấu trúc của một ngôn ngữ và cách giải thích các câu trong ngôn ngữ đó

2 Mục đích & Ý định

Mục đích chính: Định nghĩa một ngôn ngữ và cung cấp một phiên dịch viên để giải thích các câu trong ngôn ngữ đó.

Ý định:

- Tách biệt cấu trúc ngôn ngữ khỏi cách giải thích
- Dễ dàng thêm ngôn ngữ mới
- Giải quyết bài toán lặp lại

3 Vấn đề

Tình huống: Cần phân tích và thực thi một ngôn ngữ đơn giản

Vấn đề: Nếu viết trực tiếp logic phân tích, sẽ khó mở rộng khi thêm quy tắc mới

Cần một cách để biểu diễn ngôn ngữ dưới dạng cấu trúc và dễ mở rộng

4 Giải pháp

Ý tưởng: Định nghĩa các lớp cho từng phần tử ngôn ngữ

Cách làm:

- Tạo Abstract Expression cho các biểu thức
- Terminal Expression cho các phần tử cơ bản
- Non-Terminal Expression cho các quy tắc phức hợp

5 Cấu trúc

- 1. Abstract Expression:** Định nghĩa interface
`interpret(context)`
- 2. Terminal Expression:** Thực hiện phiên dịch cho ký tự/từ đơn giản
- 3. Non-Terminal Expression:** Thực hiện phiên dịch cho quy tắc phức hợp (chứa các biểu thức khác)
- 4. Context:** Chứa thông tin cần thiết cho phiên dịch

6 Luồng hoạt động

1. Phân tích chuỗi đầu vào thành cây biểu thức trừu tượng (AST)
2. Gọi `interpret()` trên gốc cây AST
3. Mỗi nút gọi `interpret()` trên nút con của nó
4. Terminal Expression trả về kết quả cuối cùng
5. Kết quả được truyền lên cây

7 Mã nguồn minh họa


```
from abc import ABC, abstractmethod
from dataclasses import dataclass
```

```
class Expression(ABC):
    @abstractmethod
    def interpret(self, context: str) -> int: pass
```

```
@dataclass
class Number(Expression):
    value: int

    def interpret(self, context: str) -> int:
        return self.value
```

```
@dataclass
class Add(Expression):
    left: Expression
    right: Expression

    def interpret(self, context: str) -> int:
        return (self.left.interpret(context) +
                self.right.interpret(context))
```

```
class Subtract(Expression):  
    def __init__(self, left, right):  
        self.left = left  
        self.right = right  
  
    def interpret(self, context: str) -> int:  
        return (self.left.interpret(context) -  
                self.right.interpret(context))
```

Sử dụng: 5 + 3 - 2

```
expr = Subtract(  
    Add(Number(5), Number(3)),  
    Number(2)
```

```
)  
result = expr.interpret("") # = 6
```

8 Ứng dụng thực tế

- **SQL Parser:** Phân tích câu lệnh SQL
- **Regular Expression:** Biểu thức chính quy
- **Compiler:** Biên dịch ngôn ngữ lập trình
- **Bộ lọc dữ liệu:** Phân tích điều kiện lọc
- **Công thức tính toán:** Excel, tính toán biểu thức

9 Trường hợp áp dụng

Sử dụng Interpreter khi:

- Cần phân tích một ngôn ngữ đơn giản
- Ngôn ngữ có thể biểu diễn bằng cây đệ quy
- Số lượng quy tắc ngôn ngữ không quá lớn
- Hiệu suất không phải ưu tiên hàng đầu

10 Ưu nhược điểm

✓ Ưu Điểm

- Dễ thêm quy tắc mới
- Tách biệt cấu trúc ngôn ngữ khỏi logic
- Dễ hiểu và bảo trì
- Tuân theo Single Responsibility

X Nhược Điểm

- Hiệu suất thấp với ngôn ngữ phức tạp
- Cây biểu thức quá lớn
- Khó debug luồng biên dịch
- Không thích hợp cho ngôn ngữ lớn

11 Cách áp dụng

1. Xác định ngôn ngữ cần phiên dịch
2. Tạo Abstract Expression interface
3. Tạo Terminal Expression cho ký tự/từ
4. Tạo Non-Terminal Expression cho quy tắc
5. Xây dựng parser để tạo cây biểu thức
6. Gọi interpret() trên gốc cây

12 So sánh với các mẫu khác

Interpreter: Phân tích và thực thi ngôn ngữ

Visitor: Duyệt qua cây đối tượng, thực hiện hành động

Strategy: Chọn chiến lược thực thi tại runtime

Composite: Tạo cây các đối tượng phức hợp

13 Các mẫu liên quan

Composite: Interpreter sử dụng cây Composite

Visitor: Có thể dùng Visitor để duyệt cây biểu thức

Iterator: Duyệt các phần tử trong cây

Factory Method: Tạo các Expression khác nhau

14 Lỗi thường gặp

Lỗi thường gặp:

- ✗ Cây biểu thức quá sâu gây stack overflow
- ✗ Không xử lý lỗi phân tích
- ✗ Context quá phức tạp

Cách khắc phục:

- ✓ Giới hạn độ sâu cây
- ✓ Thêm xử lý ngoại lệ
- ✓ Đơn giản hóa Context
- ✓ Cache kết quả (Memoization)
- ✓ Dùng regex cho ngôn ngữ đơn giản

15 Tóm tắt

Mẫu Interpreter

Định nghĩa ngôn ngữ bằng cách tạo các lớp cho mỗi quy tắc và biên dịch cây biểu thức.

Thích hợp cho: Ngôn ngữ đơn giản, DSL, Parser

Cần cẩn thận: Hiệu suất, độ phức tạp ngôn ngữ

Cải thiện: Tính mở rộng, khả năng bảo trì, tách biệt logic

Cảm ơn vì đã mua sự chú ý!

Nhóm 5